

Reasoning, sampling and hallucination

Treat every output as a likely guess until something outside the model checks it.

"Fluency is not the same as truth."

Ask on every project

1. What does one wrong output cost here, and who pays for it?
2. Which parts are exact values: numbers, names, dates, IDs or citations?
3. What catches an error before a user sees it, and does it depend on the model?
4. Where do we need the same output twice, for tests, audits or caching?
5. Would more reasoning help, or is the model missing a fact nobody gave it?

Rules of thumb

- Get numbers, prices and policy values from source systems; let the model only phrase them.
- Store outputs you must reproduce; never count on rerunning the prompt to get them back.
- Tune temperature for variety, not accuracy: a low setting can repeat a wrong answer.
- Spend reasoning on math, code and planning; for missing facts, add sources, not thinking.

Red flags

- ⚠ The only verification step is asking the model to double-check itself
- ⚠ Citations, IDs or amounts in the output with no lookup behind them
- ⚠ Quality judged from one good demo run, with no repeats and no test set
- ⚠ "A human reviews it" with no time, tooling or sample rate to make it real

Key terms

Token probability	The score the model gives each possible next token before one is picked.
Temperature	A setting that sharpens or flattens those odds before a token is sampled.
Hallucination	Fluent output that isn't supported by facts or by the sources provided.
Test-time compute	Extra tokens spent reasoning at answer time: better odds, more cost and latency.
Deterministic check	A check in code, data or rules that doesn't depend on the model's own guess.

DECISION GUIDE

Probabilistic or verified?

If variety is the point (names, drafts, ideas)	→ leave it probabilistic; test with real users
If a person reads it next to the source	→ probabilistic is OK; show or link the source
If code consumes it	→ validate the schema and the values; reject, retry or fall back
If it states a number, price, date or citation	→ fetch it from a source system; don't generate it
If it triggers an action or moves money	→ code computes and permits it; a person approves risky ones
If an error can't be caught or undone	→ don't automate it; keep a person deciding